3522 Bulut Bilişim

Akıllı Şehir Yönetim Platformu

IoT ve Akıllı Şehir Telemetri Mikroservisi

Akıllı şehir sensör verisi toplama, MQTT akışı, DynamoDB event saklama ve AWS IoT Core deployment kapsamı

Alan	Değer
Öğrenci	Ahmet Emre Özcan
Öğrenci No	21290560
Ders	3522 Bulut Bilişim
Öğretim Elemanı	Enver Bağcı
Frontend Repo Linki	https://github.com/eemrezcan/bulutproje-final-frontend
IoT Service Repo Linki	https://github.com/eemrezcan/bulutproje3-iot-service
Demo Video Linki	https://youtu.be/cwmFWNG1RY0

Kapak proje özeti

Alan	Özet
Kapsam	Proje 7 - IoT ve Akıllı Şehir Uygulaması; Akıllı şehir sensör verisi toplama, MQTT akışı, DynamoDB event saklama ve AWS IoT Core deployment kapsamı
AWS odağı	ECS Fargate service, ECR repository, DynamoDB, AWS IoT Core
Mikroservis sınırı	Ayrı API, ayrı veri modeli, ayrı deployment ve dar IAM yetki sınırı

Özet

Bu rapor, Akıllı Şehir Yönetim Platformu içinde geliştirilen IoT Service mikroservisini inceler. Servis; şehir bölgelerinden sıcaklık, nem, hava kalitesi ve trafik yoğunluğu gibi telemetry verilerini MQTT üzerinden alır, veriyi DynamoDB tablosunda zaman serisi event olarak saklar ve frontend'e REST API ile sunar. Local ortamda Mosquitto ve DynamoDB Local kullanılırken AWS ortamında AWS IoT Core, IoT Rule, Lambda, DynamoDB, ECS Fargate, ALB ve CloudFront bileşenleri ile karşılık kurulmuştur.

Anahtar Kelimeler: IoT, MQTT, AWS IoT Core, DynamoDB, FastAPI, ECS Fargate, Akıllı Şehir

İçindekiler

1 Giriş

2 Kapsam

3 Mikroservis sınırı ve sistem mimarisi

4 Teknoloji kararları

5 Veri modeli ve API tasarımı

6 Local çalışma ve demo akışı

7 AWS deployment mimarisi

8 Güvenlik, IAM ve veri sahipliği

9 Test, smoke doğrulama ve kanıtlar

10 Ortak platform entegrasyonu

11 Karşılaşılan sorunlar ve çözüm kararları

12 Sonuç

13 Kaynaklar

1. Giriş

Bu rapor, Akıllı Şehir Yönetim Platformu kapsamında geliştirilen IoT Service bileşenini ayrı bir mikroservis olarak ele alır. Platform tek bir yönetim paneli gibi kullanılsa da her mikroservis kendi veri modeline, kendi API yüzeyine, kendi container image'ına ve kendi AWS yetki sınırına sahiptir.

Temel tasarım hedefi, ders kapsamındaki bulut servislerini yalnızca isim olarak kullanmak değil, doğru problemi doğru AWS servisi ile eşleştirmektir. Bu nedenle raporda local demo akışı ile AWS deployment karşılığı birlikte anlatılmıştır.

2. Kapsam

Bu bölüm, IoT Service mikroservisinin proje içindeki sınırını ve 3522 Bulut Bilişim final yönergesindeki Proje 7 - IoT ve Akıllı Şehir Uygulaması beklentilerini nasıl karşıladığını özetler. Kapsam, yalnızca uygulama ekranını değil; veri giriş kanalını, servis sorumluluğunu, AWS karşılığını ve raporda savunulacak deployment kararlarını birlikte ele alır.

Kapsam maddesi	Projede karşılığı
Akıllı şehir sensörleri	Meydan, Otogar, Kampus, Hastane ve Sanayi bölgeleri için sentetik sensör noktaları kullanıldı.
IoT veri toplama	Sıcaklık, nem, hava kalitesi ve trafik yoğunluğu telemetry verisi üretildi.
IoT protokolü	Telemetry verisi REST ile değil MQTT topic yapısı üzerinden sisteme alındı.
Bulut karşılığı	AWS IoT Core, IoT Rule, Lambda ve DynamoDB hattı ile gerçek AWS ingestion akışı kuruldu.
Görselleştirme	React panelde son ölçümler, bölge kartları ve zaman serisi grafiği gösterildi.

Kapsam dışında bırakılan noktalar bilinçli sınırlardır. Amaç üretim ortamı ölçeğinde gerçek şehir altyapısı kurmak değil, ilgili bulut servislerini doğru mimari noktaya yerleştirerek çalışan ve anlatılabilir bir mikroservis sistemi sunmaktır.

3. Mikroservis sınırı ve sistem mimarisi

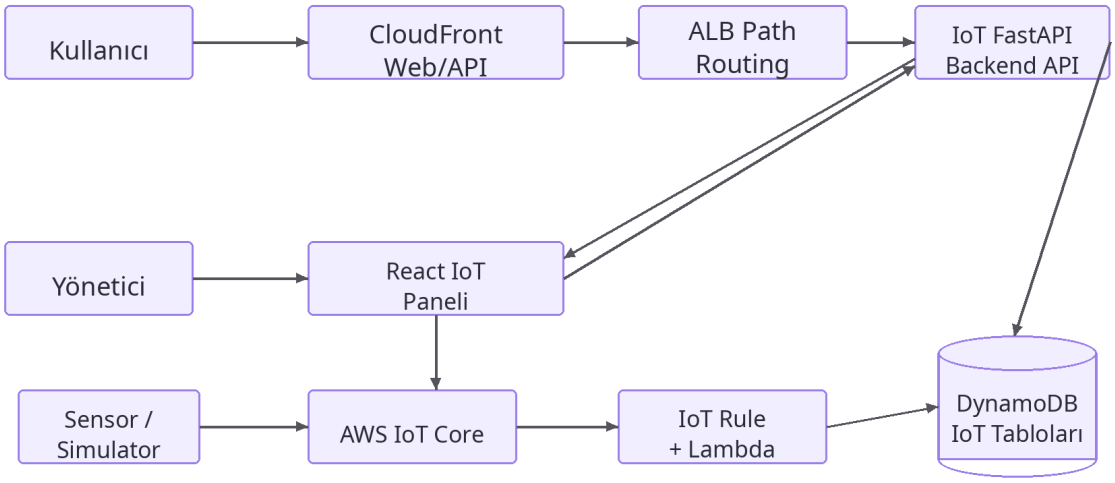
IoT Service aşağıdaki sorumlulukları üstlenir:

- Sensör cihazlarını, bölgeleri ve son telemetry durumlarını yönetir.
- MQTT topic'lerinden gelen telemetry mesajlarını işler.
- Sıcaklık, nem, hava kalitesi ve trafik yoğunluğu ölçümlerini saklar.
- Frontend için sensör listesi, son ölçüm ve bölge geçmişi API'leri sunar.

Bu servis özellikle şu görevleri üstlenmez:

- Kamera veya video akışı işlemez.
- Makine öğrenmesi tahmini üretmez.
- Video veya ML servislerinin veritabanlarına doğrudan bağlanmaz.

Ortak platformda frontend yalnızca REST API çağrıları yapar. Servisler birbirinin DynamoDB tablolarına doğrudan bağlanmaz. Bu yaklaşım mikroservis veri sahipliği ilkesini korur ve deployment ortamında IAM yetkilerinin dar tutulmasını sağlar.



Şekil 1. IoT Service mikroservis mimari diyagramı.

4. Teknoloji kararları

Bileşen	Karar ve gerekçe
Python + FastAPI	REST API, health check ve sensör okuma endpointleri için kullanıldı.
MQTT	Telemetry girişinin gerçek IoT publish/subscribe modeline benzemesi için seçildi.
Mosquitto	Local Docker Compose demosunda AWS IoT Core'un yerel karşılığıdır.
DynamoDB	Sensör metadata ve zaman damgalı ölçüm eventleri için kullanıldı.
AWS IoT Core + IoT Rule + Lambda	AWS tarafında MQTT mesajını alıp DynamoDB'ye yazan bulut hattıdır.

Backend tarafında FastAPI seçimi, hızlı REST API geliştirme, otomatik Swagger dokümantasyonu ve Python ekosistemiyle uyum nedeniyle yapılmıştır. Frontend tarafında React + Vite + TypeScript kullanılmış; canlı görünüm için TanStack Query polling tercih edilmiştir.

5. Veri modeli ve API tasarımı

iot_sensors

Alan	Açıklama
sensor_id	Partition key; sensör kimliği.
zone	Meydan, Otogar, Kampus, Hastane veya Sanayi bölgesi.
name	Kullanıcı arayüzünde görünen sensör adı.
status	active, passive veya maintenance durumu.
latitude / longitude	Harita gösterimi için konum bilgisi.

iot_sensor_readings

Alan	Açıklama
sensor_id	Partition key; ölçümün ait olduğu sensör.
timestamp	Sort key; zaman sıralı telemetry saklama.
temperature / humidity	Çevresel sensör değerleri.
air_quality_index	Hava kalitesi endeksi.
traffic_level	Trafik yoğunluğu göstergesi.
status_level	normal, warning veya critical sınıflandırması.

API yüzeyi

- GET /health
- GET /sensors
- GET /readings/latest
- GET /readings?sensor_id={sensor_id}&limit=50
- GET /zones/{zone}/readings?limit=50
- POST /simulator/start
- POST /simulator/stop

Endpoint listesi frontend'in ihtiyaç duyduğu okuma, durum ve demo kontrol akışlarını karşılar. Veri üretimi, her modülün kendi doğasına uygun giriş kanalından yapılır.

Local-AWS karşılıkları

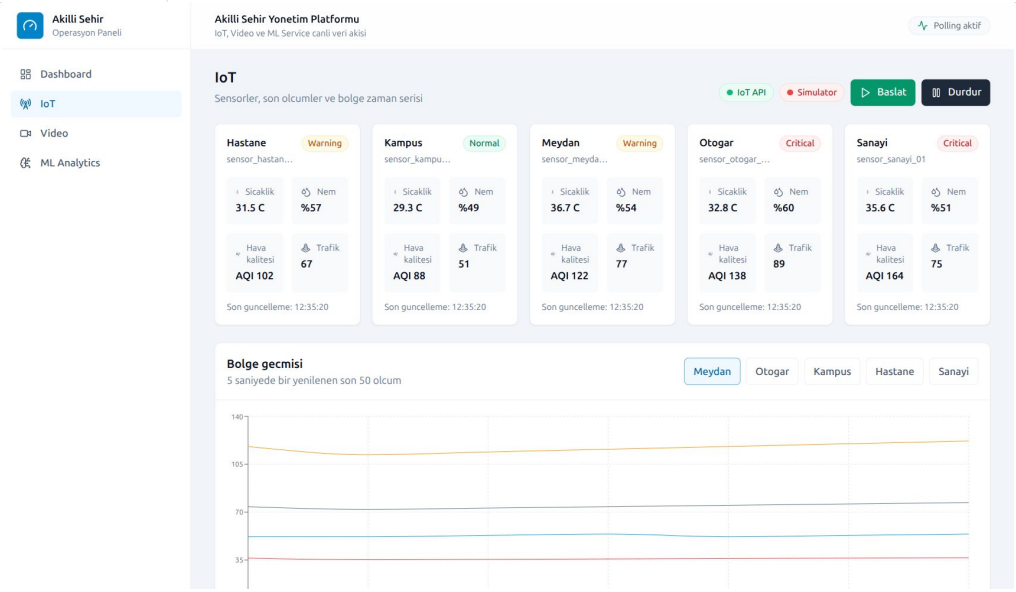
Local bileşen	AWS karşılığı
Mosquitto MQTT Broker	AWS IoT Core
IoT MQTT consumer	IoT Rule + Lambda ingestion
DynamoDB Local	Amazon DynamoDB: iot_sensors, iot_sensor_readings
FastAPI IoT container	ECS Fargate IoT service
Local simulator	IoT Core topic publish veya cihaz/simulator mesajı

6. Local çalışma ve demo akışı

Local demo Docker Compose ile çalışacak şekilde tasarlanmıştır. Frontend tek paneldir; IoT, Video ve ML servisleri ayrı container süreçleri olarak ayağa kalkar. DynamoDB Local local veritabanı karşılığıdır.

Akış adımı	Açıklama
Başlangıç	Sensor Simulator
Adım 1	MQTT publish
Adım 2	Mosquitto Broker
Adım 3	IoT Service MQTT Consumer

Akış adımı	Açıklama
Adım 4	DynamoDB Local
Adım 5	IoT Service REST API
Adım 6	Frontend IoT ekranı



Şekil 2. IoT Service canlı deployment ekran görüntüsü.

7. AWS deployment mimarisi

AWS ortamında servis, ayrı ECR repository'si, ayrı ECS Fargate service'i, ayrı task definition ve ayrı IAM task role ile konumlandırılmıştır. Frontend S3 + CloudFront üzerinden yayınlanır; `/api/*` path davranışı ALB'ye yönlendirilir.

AWS akış adımı	Açıklama
Başlangıç	Sensor / simulator
Adım 1	MQTT
Adım 2	AWS IoT Core
Adım 3	IoT Rule
Adım 4	Lambda: smart-city-final-iot-ingest
Adım 5	DynamoDB: iot_sensor_readings
Adım 6	ECS IoT API
Adım 7	CloudFront frontend

AWS kaynağı	Bu rapordaki görevi
ECS Fargate service	smart-city-final-iot-service API container'ını çalıştırır.
ECR repository	smart-city-final/iot-service image versiyonlarını saklar.
DynamoDB	iot_sensors ve iot_sensor_readings tablolarını barındırır.

AWS kaynağı	Bu rapordaki görevi
AWS IoT Core	smart-city/iot/+/+/telemetry topic'lerini MQTT giriş kanalı olarak alır.
IoT Rule + Lambda	Telemetry payload'ini doğrular ve DynamoDB'ye yazar.
CloudFront + ALB	/api/iot path'ini IoT target group'una yönlendirir.

Bu bölümdeki kaynaklar, local Docker Compose mimarisinin buluttaki karşılığıdır. Amaç backendleri yalnızca bir sunucuya koymak değil; veri girişini, analiz işini, saklama katmanını ve API yüzeyini doğru managed AWS servisleriyle ayırmaktır.

Operasyon ve maliyet notları

Konu	Not
Geçici kapatma	ECS service desired-count=0 yapılarak Fargate task maliyeti durdurulabilir.
Veri koruma	DynamoDB, S3, ECR ve IAM kaynakları silinmeden demo verileri korunur.
ALB notu	ALB'nin stop modu yoktur; tamamen maliyeti kesmek için silmek gerekir, demo öncesi önerilmez.
Yaklaşık maliyet	3 Fargate task + ALB + public IPv4 düşük trafikte yaklaşık 0.13 USD/saat düzeyindedir.

8. Güvenlik, IAM ve veri sahipliği

Her mikroservise yalnızca kendi verisine erişim yetkisi verilmiştir. ECS task execution role sadece ECR image çekme ve CloudWatch Logs yazma gibi altyapı görevleri için; uygulama task role ise servis özelindeki DynamoDB/S3/Lambda erişimleri için kullanılır.

Bu ayrım, mikroservis sınırını hem kod hem deployment seviyesinde görünür hale getirir. Bir servisin diğer modülün tablosuna doğrudan erişmemesi, hata izolasyonu ve ilerde ölçekleme açısından daha sağlıklı bir tasarımdır.

9. Test, smoke doğrulama ve kanıtlar

Kontrol	Sonuç
CloudFront IoT API health	/api/iot/health status=ok döndürdü.
IoT readings	/api/iot/readings/latest beş bölge için son ölçümleri döndürdü.
IoT Core test	smart-city/iot/meydan/sensor_meydan_01/telemetry topic'ine publish edilen mesaj DynamoDB'ye yazıldı.
Frontend smoke	IoT ekranında Hastane, Kampus, Meydan, Otogar ve Sanayi kartları göründü.
ECS service	desired=1, running=1, target health=healthy.

Canlı frontend URL'si https://d1k4ey8nhx4dtb.cloudfront.net adresidir.

Bu doğrulamalar teslim videosunda anlatılacak akış için de kanıt niteliğindedir: önce canlı frontend açılır, ardından modül ekranı gösterilir, en son API ve AWS kaynak eşleşmesi açıklanır.

10. Ortak platform entegrasyonu

Bu mikroservis tek başına anlamlı bir modüldür; ancak platform değeri frontend'de diğer servislerle birlikte görünür. IoT verileri şehir durumunu, video analizleri kamera yoğunluğunu, ML servisi ise risk ve öneri katmanını üretir.

Ortak frontend'in veritabanı yoktur. API Gateway kullanılmamış, bunun yerine CloudFront + ALB path routing ile sade ve gösterilebilir bir deployment kurgusu kurulmuştur.

11. Karşılaşılan sorunlar ve çözüm kararları

Sorun / karar alanı	Çözüm
Local ve AWS MQTT ayrımı	Localde Mosquitto kullanıldı; AWS'te MQTT hattı IoT Core + Rule + Lambda ile temsil edildi.
Servis verisi sahipliği	ML veya Video servisleri IoT tablolarına doğrudan bağlanmadı; veri REST API ile paylaşıldı.
Demo kararlılığı	Sentetik sensör üretimi kontrollü değer aralıklarıyla yapıldı ve beş bölge için stabil veri sağlandı.

12. Sonuç

IoT Service, Proje 7 kapsamındaki MQTT tabanlı cihaz haberleşmesi, bulut veri işleme ve gerçek zamanlıya yakın görselleştirme beklentilerini karşılar. AWS tarafında IoT Core + Rule + Lambda + DynamoDB hattı kurulmuş, API katmanı ise ayrı ECS Fargate servisi olarak deploy edilmiştir.

Kaynaklar

Kaynaklar: 3522 Bulut Bilişim final proje yönergesi; AWS resmi dokümantasyonu; FastAPI, React/Vite/TanStack Query/Recharts ve AWS IoT Core ve Mosquitto dokümantasyonları; proje repository dokümanları.